## Instrucción

El presente documento describe el diseño arquitectónico y el modelo de diseño orientado a objetos para un Sistema de Inventario pensado para empresas medianas con múltiples almacenes, control de lotes y necesidad de integración con puntos de venta y sistemas contables. Se parte de los fundamentos de la arquitectura de software y del análisis orientado a objetos para transformar requisitos reales de usuarios en una solución coherente, modular y preparada para la evolución tecnológica.

El objetivo es ofrecer un anteproyecto técnico que sirva como guía para la construcción: especifica los requisitos funcionales y no funcionales, propone una arquitectura híbrida cliente-servidor con servicios modulares y describe las principales entidades del dominio mediante diagramas y modelos de clases. Este enfoque busca equilibrar la resolución de las necesidades inmediatas del negocio con la flexibilidad necesaria para adaptarse a cambios futuros, garantizando mantenibilidad, seguridad y escalabilidad.

El documento incluye además ejemplos de interacción (casos de uso), patrones de diseño orientado a objetos aplicables y consideraciones para una posible migración a microservicios. Está pensado para ser utilizado por desarrolladores, arquitectos y responsables de producto como base para la implementación, las pruebas y la planificación de despliegues.

**Sistema de Inventario**

**Descripción breve** Aplicación para gestionar productos, existencias, entradas y salidas, proveedores, órdenes de compra y reportes en tiempo real para una o varias bodegas.

**Dominio de aplicación y alcance**

**Dominio elegido Sistema de Inventario** para empresas medianas que manejan múltiples almacenes, control de lotes y fechas de vencimiento, y requieren integración con punto de venta y compras.

**Alcance funcional mínimo entregable**

- ✓ Gestión de productos y categorías
- ✓ Control de existencias por almacén y por lote
- ✓ Registro de entradas y salidas (recepciones, ventas, ajustes)
- ✓ Gestión de proveedores y órdenes de compra
- ✓ Generación de reportes e indicadores (stock, rotación, alertas de bajo stock)
- ✓ Autenticación y control de roles y permisos
- ✓ API para integración con POS y sistemas contables

**2-Enumera los requisitos funcionales y no funcionales que la aplicación debe cumplir. Incluye las funciones principales, el rendimiento esperado, la seguridad, la escalabilidad y otros aspectos relevantes.**

**Requisitos funcionales principales**

1. **RF01 Registrar producto**: Crear/editar/eliminar productos con código, descripción, unidad, categoría, precio, proveedor preferente, lote y fecha de vencimiento.
2. **RF02 Gestión de almacenes**: Crear múltiples almacenes; consultar stock por almacén.
3. **RF03 Movimientos de inventario**: Registrar entradas (recepciones), salidas (ventas, transferencias), ajustes y devoluciones.

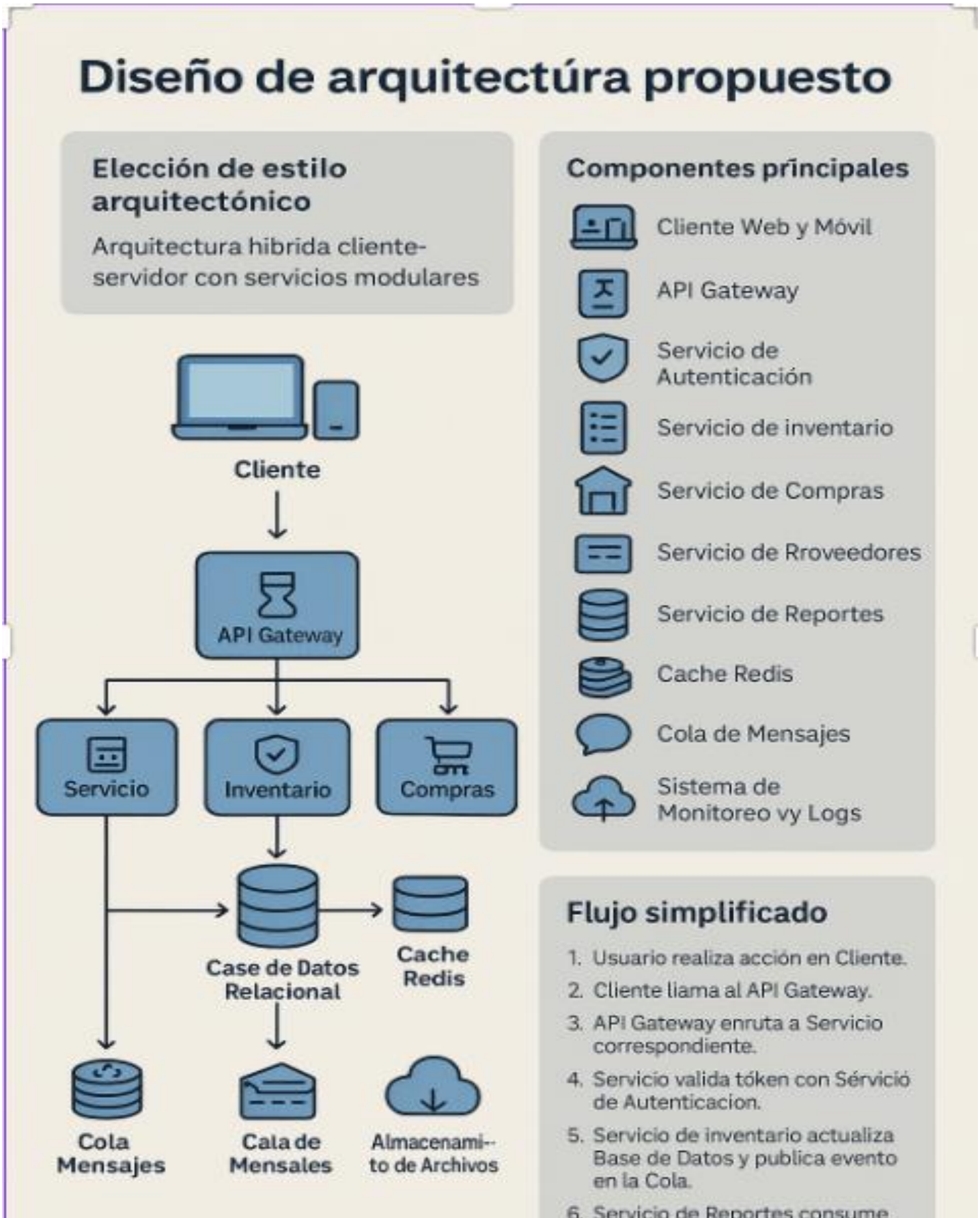
4. **RF04 Órdenes de compra**: Generar, aprobar y recibir órdenes de compra; actualizar stock al recibir.
5. **RF05 Transferencias entre almacenes**: Solicitar y completar transferencias internas.
6. **RF06 Alertas y notificaciones**: Alertas por bajo stock, vencimiento de lotes y recepción pendiente.
7. **RF07 Reportes**: Reportes de stock, historial de movimientos, rotación de inventario, valor de inventario.
8. **RF08 Autenticación y autorización**: Login, roles (admin, almacén, comprador, auditor) y permisos por acción.
9. **RF09 API REST**: Endpoints para consultar stock, registrar movimientos y sincronizar con POS.
10. **RF10 Auditoría**: Registro inmutable de transacciones de inventario con usuario y timestamp.

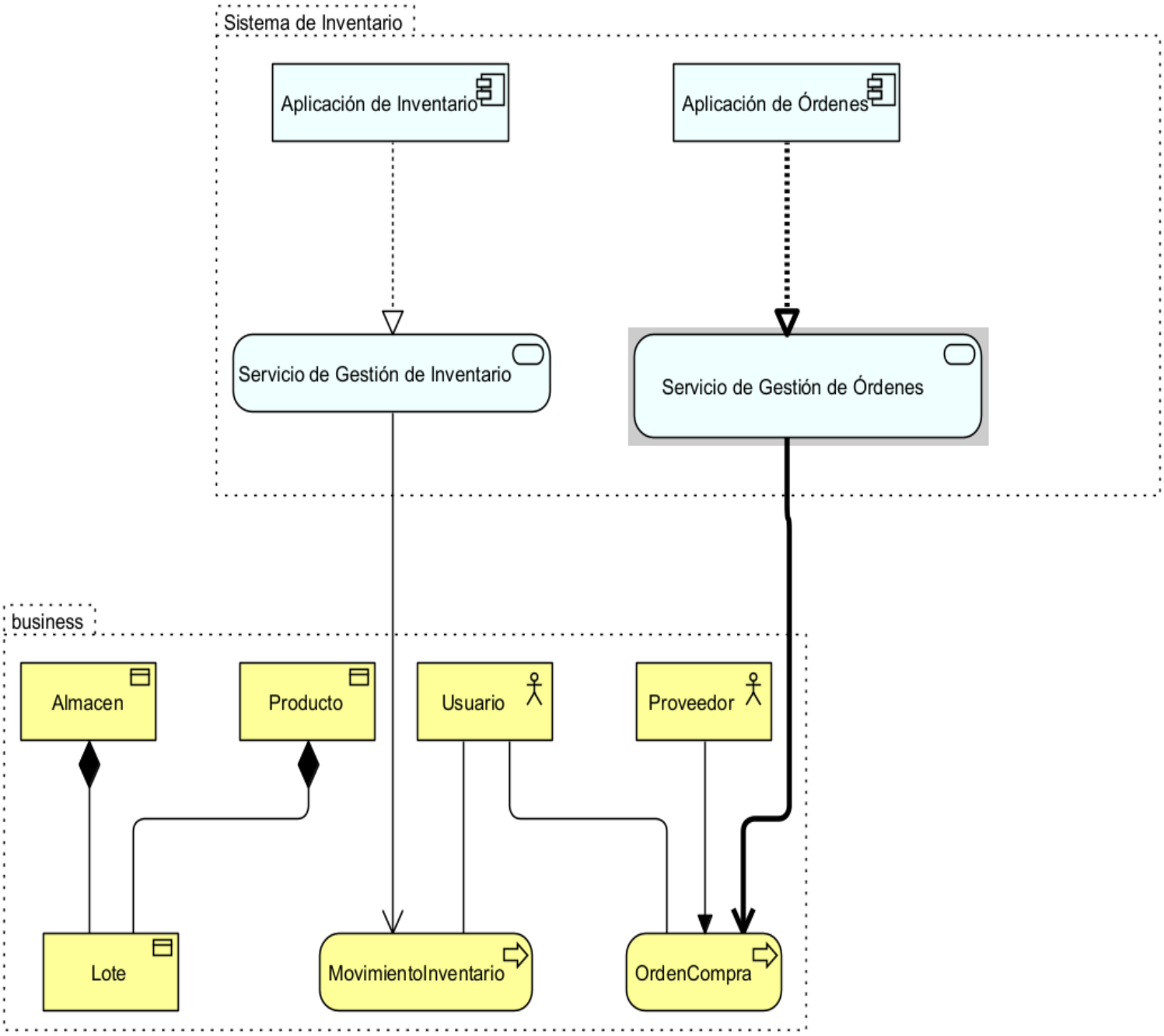
**Requisitos no funcionales**

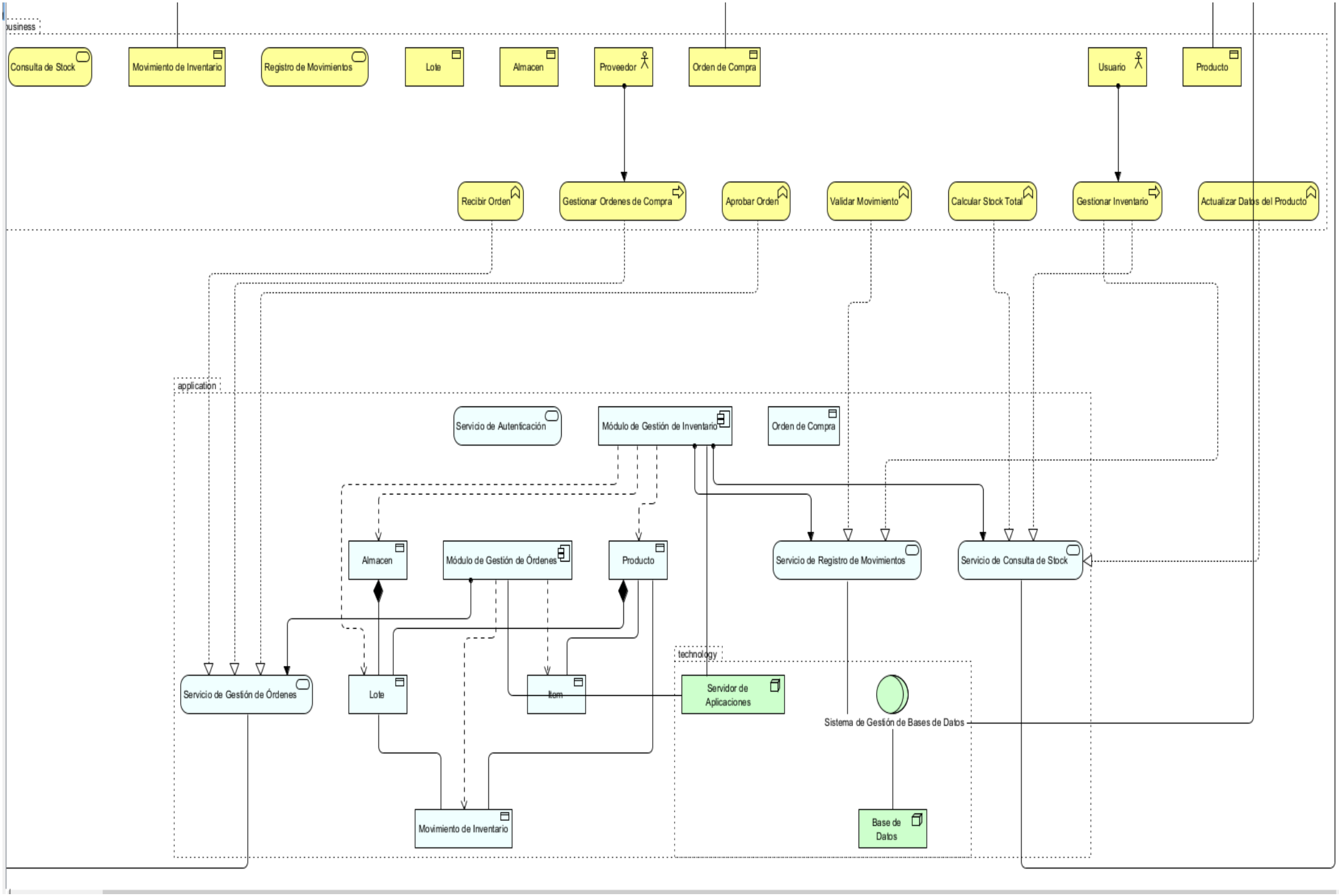
1. **Rendimiento**: Respuesta < 300 ms para consultas de stock en condiciones normales; capacidad para 200 transacciones concurrentes en picos.
2. **Disponibilidad**: 99.9% SLA para la API crítica; ventanas de mantenimiento planificadas.
3. **Escalabilidad**: Escalado horizontal de la capa de aplicación y de la base de datos mediante réplicas y particionado por almacén si es necesario.
4. **Seguridad**: Autenticación basada en tokens JWT; cifrado TLS en tránsito; cifrado de datos sensibles en reposo; control de acceso por roles.
5. **Consistencia**: Consistencia eventual aceptable para reportes agregados; consistencia fuerte para operaciones de decremento de stock (reservas y ventas).
6. **Mantenibilidad**: Código modular, pruebas unitarias y de integración, documentación de API y diagramas de arquitectura.
7. **Portabilidad**: Contenedores Docker para despliegue en nubes públicas o privadas.
8. **Observabilidad**: Logs estructurados, métricas (Prometheus) y trazas distribuidas (OpenTelemetry).
9. **Internacionalización**: Soporte para múltiples idiomas y formatos de fecha/moneda.

10. **Backup y recuperación**: Backups diarios y plan de recuperación con RTO < 2 horas y RPO < 1 hora para datos críticos.

**3-Representa el diseño propuesto utilizando un diagrama de arquitectura (cliente-servidor, P2P, objetos distribuidos, o híbrido). Puedes usar herramientas como Visual Paradigm , Enterprise Arquitect u otra herramienta profesional de modelado**




Sistema de Inventario
Aplicación de Inventario
Aplicación de Órdenes
Servicio de Gestión de Inventario
Servicio de Gestión de Órdenes
business
Almacen
Producto
Usuario
Proveedor
Lote
MovimientoInventario
OrdenCompra


business
Consulta de Stock
Movimiento de Inventario
Registro de Movimientos
Lote
Almacen
Proveedor
Orden de Compra
Usuario
Producto
Recibir Orden
Gestionar Ordenes de Compra
Aprobar Orden
Validar Movimiento
Calcular Stock Total
Gestionar Inventario
Actualizar Datos del Producto
application
Servicio de Autenticación
Módulo de Gestión de Inventario
Orden de Compra
Almacen
Módulo de Gestión de Órdenes
Producto
Servicio de Registro de Movimientos
Servicio de Consulta de Stock
technology
Servicio de Gestión de Órdenes
Lote
Item
Servidor de Aplicaciones
Sistema de Gestión de Bases de Datos
Movimiento de Inventario
Base de Datos

## 3 Diseño de arquitectura propuesto

### Elección de estilo arquitectónico

### Arquitectura híbrida cliente servidor con servicios modulares

- ✓ **Cliente**: Aplicación web SPA (React o Angular) y cliente móvil ligero.
- ✓ **Servidor**: Backend RESTful monolítico modular o conjunto de servicios desacoplados (híbrido preparado para migrar a microservicios).
- ✓ **Persistencia**: Base de datos relacional (PostgreSQL) para transacciones; caché Redis para consultas frecuentes; almacenamiento de archivos en S3 compatible.
- ✓ **Integraciones**: API Gateway para exponer endpoints; colas (RabbitMQ o Kafka) para procesos asíncronos (notificaciones, sincronización con POS).

### Diagrama de arquitectura textual listo para copiar en Word

### Componentes principales

- ✓ **Cliente Web y Móvil**
- ✓ **API Gateway**
- ✓ **Servicio de Autenticación**
- ✓ **Servicio de Inventario**
- ✓ **Servicio de Compras**
- ✓ **Servicio de Proveedores**
- ✓ **Servicio de Reportes**
- ✓ **Base de Datos Relacional**
- ✓ **Cache Redis**
- ✓ **Cola de Mensajes**
- ✓ **Almacenamiento de Archivos**
- ✓ **Sistema de Monitoreo y Logs**

### Flujo simplificado

1. Usuario realiza acción en Cliente.
2. Cliente llama al API Gateway.
3. API Gateway enruta a Servicio correspondiente.
4. Servicio valida token con Servicio de Autenticación.

5. Servicio de Inventario actualiza Base de Datos y publica evento en la Cola.
6. Servicio de Reportes consume eventos para actualizar agregados en cache o almacén analítico.
7. Notificaciones se envían de forma asíncrona.

**Flujo de UML**

**Producto**

| Clase | Atributos | Métodos |
|---|---|---|
| **Producto** | **id; sku; nombre; descripción; unidad; categoríaId; precio; proveedorId** | **actualizarDatos(); calcularStockTotal()** |

**Almacén**

| Clase | Atributos | Métodos |
|---|---|---|
| **Almacén** | **id; nombre; ubicación; capacidad** | **consultarStock(productoId)** |

**Lote**

| Clase | Atributos | Métodos |
|---|---|---|
| **Lote** | **id; productoId; numeroLote; fechaVencimiento; cantidadDisponible** | **reservar(cantidad); liberar(cantidad)** |

**MovimientoInventario**

| Clase | Atributos | Métodos |
|---|---|---|
| **MovimientoInventario** | **id; tipo; productoId; loteId; cantidad; almacenOrigen; almacenDestino; usuarioId; timestamp** | **validar(); aplicar()** |

**OrdenCompra**

| Clase | Atributos | Métodos |
|---|---|---|
| OrdenCompra | id; proveedorId; items; estado; fechaCreacion; fechaRecepcion | aprobar(); recibir() |

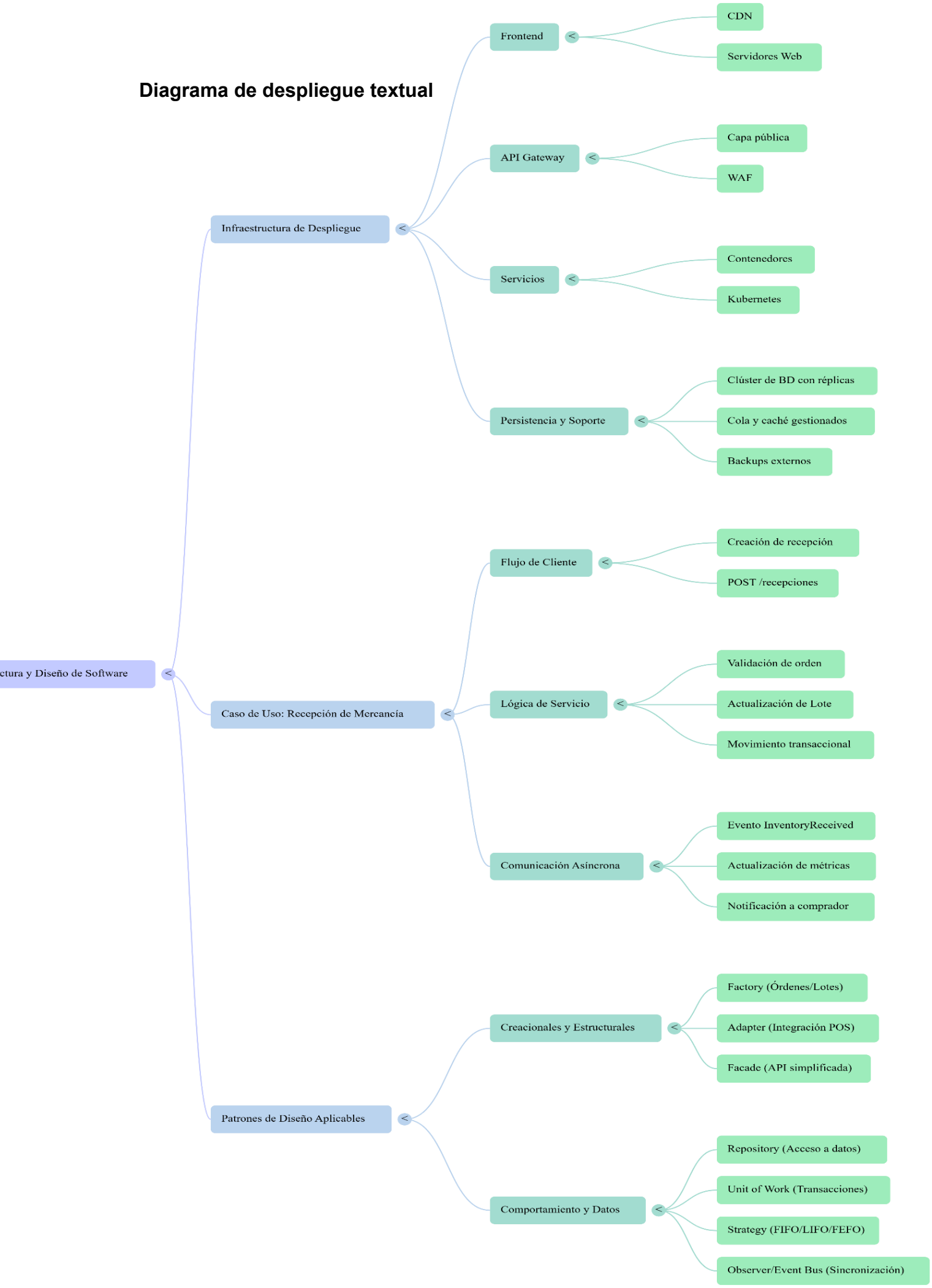
**Proveedor**

| Clase | Atributos | Métodos |
|---|---|---|
| Proveedor | id; nombre; contacto; condicionesPago | crearOrdenCompra() |

**Usuario**

| Clase | Atributos | Métodos |
|---|---|---|
| Usuario | id; nombre; email; rol | autenticar(); autorizar(accion) |

# Diagrama de despliegue textual



**Preguntas de reflexión:**

**Cómo afecta el diseño de la arquitectura a la mantenibilidad y escalabilidad de la aplicación**

- ✓ **Mantenibilidad** mejora con modularidad: separar responsabilidades en servicios y capas (API, lógica de negocio, persistencia) facilita pruebas, despliegues y cambios localizados. Código bien documentado y patrones claros reducen la deuda técnica.
- ✓ **Escalabilidad** se facilita con separación de componentes: escalar horizontalmente la capa de aplicación y cachear consultas frecuentes reduce carga en la base de datos. Diseñar límites de contexto claros (bounded contexts) evita acoplamientos que impidan escalar partes independientes.

**Qué patrones de diseño orientado a objetos podrían aplicarse en tu diseño de arquitectura**

- ✓ **Repository, Factory, Strategy, Unit of Work, Observer, Adapter, Facade**. Estos patrones ayudan a desacoplar, facilitar pruebas unitarias, soportar múltiples políticas de negocio y conectar con sistemas externos sin romper el núcleo del dominio.

**Cómo se pueden abordar los requisitos de seguridad y rendimiento en el diseño**

- ✓ **Seguridad**: autenticación centralizada (OAuth2/JWT), autorización por roles, cifrado TLS, validación de entrada, logging de auditoría, pruebas de penetración y gestión de secretos.
- ✓ **Rendimiento**: uso de cache (Redis), índices y consultas optimizadas en BD, colas para operaciones asíncronas, paginación en endpoints, limitación de tasa (rate limiting) en API Gateway y pruebas de carga para dimensionar recursos.

**Consideraciones al migrar esta arquitectura a microservicios**

- ✓ **Descomposición por dominio**: identificar bounded contexts (Inventario, Compras, Proveedores, Reportes).
- ✓ **Gestión de datos**: cada microservicio con su propia base de datos para evitar acoplamientos; usar eventos para sincronización.
- ✓ **Transacciones distribuidas**: evitar transacciones distribuidas; usar sagas o compensaciones para mantener consistencia.
- ✓ **Observabilidad**: trazabilidad distribuida, métricas y alertas por servicio.

- ✓ **Operaciones**: automatización CI/CD, despliegue canary, gestión de configuración y secretos.
- ✓ **Costos y complejidad**: evaluar sobrecarga operativa y decidir si la escala y la necesidad de despliegue independiente justifican la migración.

**Bibliografía**


- ✓ Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1995). *Design patterns: Elements of reusable object-oriented software* (1.ª ed.). Addison-Wesley.
- ✓ Fowler, M. (2002). *Patterns of enterprise application architecture*. Addison-Wesley.
- ✓ Evans, E. (2003). *Domain-driven design: Tackling complexity in the heart of software*. Addison-Wesley.
- ✓ Richardson, C. (2018). *Microservices patterns: With examples in Java*. Manning Publications.
- ✓ Fielding, R. T. (2000). *Architectural styles and the design of network-based software architectures* (Doctoral dissertation). University of California, Irvine. https://www.ics.uci.edu/~fielding/pubs/dissertation/top.htm (ics.uci.edu in Bing)
- ✓ Jones, M., Bradley, J., & Sakimura, N. (2015). *JSON Web Token (JWT)*. RFC 7519. Internet Engineering Task Force. https://doi.org/10.17487/RFC7519
- ✓ Hardt, D. (2012). *The OAuth 2.0 authorization framework*. RFC 6749. Internet Engineering Task Force. https://doi.org/10.17487/RFC6749
- ✓ Rescorla, E. (2001). *SSL and TLS: Designing and building secure systems*. Addison-Wesley.
- ✓ Burns, B., Grant, B., Oppenheimer, D., Brewer, E., & Wilkes, J. (2016). *Borg, Omega, and Kubernetes*. *Communications of the ACM, 59*(5), 50–57. https://doi.org/10.1145/2890784
- ✓ Hightower, K., Burns, B., & Beda, J. (2017). *Kubernetes: Up and running: Dive into the future of infrastructure* (2.ª ed.). O'Reilly Media.
- ✓ Merkel, J. (2014). *Docker: Lightweight Linux containers for consistent development and deployment*. *Linux Journal, 2014*(239), Article 2.
- ✓ Matthias, K., & Kane, S. P. (2015). *Docker: Up & Running: Shipping reliable containers in production*. O'Reilly Media.

- ✓ Narkhede, N., Shapira, G., & Palino, T. (2017). *Kafka: The definitive guide: Real-time data and stream processing at scale*. O'Reilly Media.
- ✓ Videla, A., & Williams, J. J. W. (2012). *RabbitMQ in action*. Manning Publications.
- ✓ Brazil, B. (2018). *Prometheus: Up & running: Infrastructure and application performance monitoring*. O'Reilly Media.
- ✓ The PostgreSQL Global Development Group. (2024). *PostgreSQL documentation* (versión 15). https://www.postgresql.org/docs/
- ✓ The OpenTelemetry Authors. (2024). *OpenTelemetry specification*. Cloud Native Computing Foundation. https://opentelemetry.io
- ✓ Bass, L., Clements, P., & Kazman, R. (2012). *Software architecture in practice* (3.ª ed.). Addison-Wesley.
- ✓ Buschmann, F., Meunier, R., Rohnert, H., Sommerlad, P., & Stal, M. (1996). *Pattern-oriented software architecture: A system of patterns* (Vol. 1). Wiley.
- ✓ Fowler, M., & Lewis, J. (2014). *Microservices: A definition of this new architectural term*. martinfowler.com. . https://martinfowler.com/articles/microservices.html (martinfowler.com in Bing)